

HW 2 - Report

TASK 0: Lab Preparation

I created the Python virtual environment and the required libraries.

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ python3 --version
Python 3.12.3
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ pip show cryptography
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/nweiss-csce465/Documents/csce465/HW2/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
```

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ pip show pytest
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/nweiss-csce465/Documents/csce465/HW2/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
```

I also generated the necessary ffdhe3072 parameter file for Task 2.

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
```

TASK 1: Demonstrating Why Encryption Alone is Insufficient

For Task 1, I set up an AES-CTR encryption program specifically for the command ‘{“action”:“READ”,”path”: “notes.text”}’ without using a MAC. In the program, a relay modifies the ciphertext by XORing the bytes that correspond to ‘READ’ with the difference between ‘READ’ and ‘SEND’. As a result, once the receiver decrypts the modified ciphertext, “SEND” is processed instead of “READ”. Additionally, I replayed the original ciphertext twice, and the receiver processed it both times. The reason “SEND” was processed is because encryption does not authenticate ciphertext on its own. Moreover, the original ciphertext was processed twice because encryption does not know if a message has been processed before.

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ python
baseline_ctr.py
Original plaintext: b'{"action":"READ","path":"notes.txt"}'
Original ciphertext: cb3f126782f0e3ace85d3d02b0b11ed129ef54d00e2ee46186ed9cd6e60
6df8df75debe0

Original bytes: b'READ'
Modified bytes: b'SEND'
XOR mask: 01000f00
Modified ciphertext: cb3f126782f0e3ace85d3d03b0be1ed129ef54d00e2ee46186ed9cd6e60
6df8df75debe0

--- Modified Message ---
Receiver processed: b'{"action":"SEND","path":"notes.txt"}'

--- Replay Demonstration ---
Receiver processed: b'{"action":"READ","path":"notes.txt"}'
Receiver processed: b'{"action":"READ","path":"notes.txt"}'
```

TASK 2: Authenticated Finite-Field Diffie–Hellman Setup

For this task, I implemented a handshake between a simulated gateway and node. Each party has an RSA signing key that lasts long-term while new Diffie-Hellman keys and nonces are generated at the start of each session. The gateway and node both create the same transcript containing what is important from the handshake (identities, DH public values, nonces, and group being used). Because this information is signed, it helps prevent someone in the middle from changing the DH values since the signatures would then no longer be verified. Additionally, the roles being included helps prevent reflection attacks, since it informs the party receiving the reflected message that the message was not sent by the expected party.

After the handshake is verified, both parties create the same shared secret and pass it into the KDF to create the separate encryption and MAC keys. Since the DH keys are created each session, it provides forward secrecy, while the nonces make each handshake different, which makes it harder to perform a replay attack.

If the RSA signatures were not included, an attacker could easily hide between the gateway and node and replace the DH values without either party realizing. Then, while both parties think they are talking to each other, the attacker could create a shared secret with the gateway and then create another with the node.

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ python
handshake.py
Handshake successful
Session ID: 8927cef93458a180
```

TASK 3: Encrypt-then-MAC Record Layer

For Task 3, I used the keys from the handshake to securely send messages between the gateway and node. AES-CTR was used to encrypt the message, while HMAC-SHA-256 was used to ensure that the encrypted message was not changed before being decrypted. Additionally, different keys are used for each direction, and each message comes with a sequence number. This helps prevent old messages from being reused as an attack.

Here is a screenshot of output from me testing 'secure_record.py' while implementing it:

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ python
secure_record.py
Gateway to node:
Message type: 1
Plaintext: b'Hello from gateway'

Node to gateway:
Message type: 1
Plaintext: b'Hello from node'
```

TASK 4: Adversarial Tests and Security Note

For the final task, I established tests to make sure that any issues that occur with handshakes or messages fail safely. The tests check cases where ciphertext or message information is changed, where messages are replayed, or where messages are sent in the wrong direction. There is also a test checking the case where there is an invalid handshake authentication. Additionally, a valid handshake with normal messages in both directions are tested to make sure that the system works properly.

Here is a screenshot showing all eight automated tests passing:

```
(.venv) nweiss-csce465@nweiss-csce465-VirtualBox:~/Documents/csce465/HW2$ python
-m pytest -q
..... [100%]
8 passed in 2.98s
```

Security Note:

For this application, the attacker I am considering the most is someone who is sitting somewhere between the gateway and the node. They may be able to observe, change, delay, or resend data and/or messages that are being sent between the two parties. The goal is to stop the attacker from being able to actually read the messages being sent in plaintext. The application is not meant to protect against attackers who already have access to private information, keys, or the system itself.

CTR mode works by generating a stream of data that will be combined with the plaintext to turn it into ciphertext. Because of this, the IV needs to be unique each time the same key is used. The IV is what makes sure that encrypting different messages does not produce the same output. If the same IV is reused, the same stream would be generated again, which could potentially allow an attacker to gain useful information if they were to compare the messages. Even if the attacker could not fully figure out the plaintext immediately, reusing the same stream could still reveal patterns or relationships between the messages.

Verifying that a message has not been altered in any way before actually opening/decrypting the message is important because a message that has been changed could be processed before the system has the chance to realize that something is wrong. Using a tool (like HMAC) before actually decrypting the message to ensure its validity prevents potentially harmful data from being consumed by the application. This also gives the receiver a way to reject the message early instead of continuing to process information that should not be trusted.

It's important to separate encryption and MAC keys because reusing the same key for both would significantly increase the overall attack surface available for an attacker. If there was some issue/breach occurring with one use of the key, then that issue could also have an affect on the other use of the key. By separating the keys, we limit the damage that could happen if one of them is compromised. Additionally, it keeps the purpose of each key separate, rather than relying fully on a single key for multiple parts of the security process.

This design uses AES-CTR together with HMAC, while an AEAD mode (such as AES-GCM) combines encryption and authentication all into one. Essentially, both methods are meant to provide both confidentiality and integrity, but this application has encryption and authentication as two different steps instead of combining them like AES-GCM.

Even though a record may be protected and its integrity remains intact, that does not automatically mean that what the record contains is safe. The encryption and HMAC used by the application simply make sure that the messages come through a protected channel without being altered. They do not actually determine whether the contents of the message are safe. If an authorized user sends a valid message that contains harmful information or instruction, the message would still be accepted. There would have to be some additional checks to decide whether the message is safe and acceptable to process.